

A Hybrid Metaheuristic Algorithm for Multiobjective Scheduling in Distributed Heterogeneous Flexible Job Shops

Lijun Wang , Chengguang Wang , Xiangyang Li , Ruixue Cheng , Xiao Ma, and Dong Liu

Abstract—To address the issues of insufficient population diversity and the tendency to fall into local convergence in the NSGA-II algorithm for distributed heterogeneous flexible job shop scheduling, a dual-population improved algorithm, D3QN-NSGA-II, based on D3QN is proposed. By introducing a dual-population evolution strategy, the breeding population and the regular population are divided using a reproductive capacity assessment mechanism, with differentiated crossover and mutation operators designed for each. A hybrid strategy, consisting of 12 types of local search operators across 3 categories, is constructed, and the innovative application of D3QN deep reinforcement learning for dynamic operator selection is introduced. Additionally, an energy-efficient scheduling strategy is applied to optimize machine idle time. Finally, the effectiveness of the D3QN-NSGA-II algorithm is evaluated using 20 instances, with experimental results demonstrating its superiority over several other advanced algorithms in solving the DHFJS problem.

Index Terms—Distributed manufacturing, D3QN, metaheuristic algorithms, modeling simulation, optimal scheduling.

I. INTRODUCTION

WITH the increasing globalization and diversification of market demands, the manufacturing industry faces mounting challenges. These challenges, particularly in production scheduling and energy efficiency, have driven the development of advanced manufacturing technologies and systems. In this context, distributed flexible job shops have emerged as

Received 23 March 2025; revised 29 June 2025; accepted 18 August 2025. Date of publication 11 September 2025; date of current version 3 April 2026. This work was supported in part by the Foreign Expert Project of Ministry of Science and Technology of the People's Republic of China under Grant G2023026004L, in part by the Graduate Education Reform Project of Henan Province under Grant 2023SJGLX021Y, and in part by the Fund of Innovative Education Program for Ph.D. Graduate Students with the North China University of Water Resources and Electric Power under Grant NCWUBC202326. (Corresponding author: Lijun Wang.)

Lijun Wang, Xiao Ma, and Dong Liu are with the School of Mechanical Engineering, North China University of Water Resources and Electric Power, Zhengzhou 450045, China (e-mail: wljb@163.com; Maxiao@ncwu.edu.cn; 2501699414@qq.com).

Chengguang Wang and Xiangyang Li are with the School of Management and Economics, North China University of Water Resources and Electric Power, Zhengzhou 450045, China (e-mail: 201709502@stu.ncwu.edu.cn; lxy1231@163.com).

Ruixue Cheng is with the Centre for Sustainable Engineering, Teesside University, TS1 3BA Middlesbrough, U.K. (e-mail: r.cheng@tees.ac.uk).

Digital Object Identifier 10.1109/TCSS.2025.3601624

a promising production system model, demonstrating significant potential in enhancing production flexibility and efficiency. However, the scheduling problem in distributed heterogeneous flexible job shops is both complex and challenging. It not only involves the allocation of jobs, the determination of production sequences, and machine selection, but also requires the minimization of energy consumption while maintaining production efficiency. The complexity of these issues renders traditional scheduling methods inadequate to meet the evolving demands of modern manufacturing, particularly in the face of rising energy costs and stricter environmental regulations. Therefore, researching and developing new scheduling algorithms to achieve dual optimization of production efficiency and energy utilization is of paramount theoretical and practical significance for the sustainable development of the manufacturing industry.

The distributed heterogeneous flexible job shop (DHFJS) represents a significant and intricate extension of the traditional flexible job shop (FJS) model. The system can be divided into several principal components, as follows. The three key processes that must be considered are the allocation of jobs to various factories, the determination of production sequences within each factory, and the selection of suitable machines for each operation. Given its inherent complexity, the DHFJS is classified as an NP-hard problem. In addressing this challenge, metaheuristic algorithms, including grey wolf optimization, particle swarm optimization, and genetic algorithms, are frequently utilized. Despite traditional assumptions that each factory has identical machine types and processing capabilities, practical scenarios often feature heterogeneity among factories due to constraints such as cost, space, and worker skills. In practical distributed manufacturing scenarios, factories often display heterogeneity, characterized by variations in the number, quality and processing times of machines.

The scheduling problem of distributed job shops has attracted considerable interest from scholars at home and abroad, given the widespread application of flexible manufacturing systems in distributed manufacturing plants. Luo et al. [1] proposed an efficient memetic algorithm for DFJSS with transfers, emphasizing the importance of efficient algorithms in solving such problems. Meng et al. [2] focused on energy-efficient DFJSS and introduced a hybrid shuffled frog-leaping algorithm for large-sized problems. Chen et al. [3] proposed a multiobjective mathematical model to minimize the total weighted delay and total

energy consumption in low-carbon heterogeneous distributed flexible job shops, where dynamic intelligent scheduling can significantly improve production efficiency. Jia et al. [4] introduced a multiobjective brainstorming optimization method to solve the integrated problem of distributed flexible job shop and delivery. Wang et al. [5] developed a multiobjective whale optimization algorithm that optimizes both energy consumption and makespan in distributed welding shop scheduling. Tang et al. [6] developed a hybrid optimization algorithm based on teaching and learning to solve the problem of distributed sand mold casting shop scheduling. Zhu et al. [7], [8] tackled the dynamic distributed flexible job shop scheduling problem by considering operation inspection and order cancellation. Pan et al. [9] proposed a knowledge-based two-population optimization (KTPO) algorithm to solve the distributed energy-efficient parallel machine scheduling problem (DEPMSP) with the objective of minimizing the total energy consumption and total delay. However, these studies, primarily relying on traditional heuristics or metaheuristics, generally exhibit limited adaptability to dynamic production environments and lack the end-to-end decision-making capabilities required for complex scheduling scenarios, failing to effectively balance global exploration and local exploitation in multiobjective optimization.

Deep reinforcement learning represents a fusion of the feature extraction capabilities of deep learning with the decision-making abilities of reinforcement learning. It employs environmental data as input to obtain experiences of success or failure, which are then utilized to update the parameters of the decision network, thereby facilitating optimal decision-making [10], [11], [12], [13]. Wang et al. [14] proposed a double-Q learning (D-Q) method for the uncertainty of the production environment in the assembly job shop, combined with the real-time characteristics of reinforcement learning. Zhao et al. [15] proposed a coevolutionary algorithm with a duel reinforcement learning mechanism for the distributed heterogeneous flexible flow shop scheduling problem. Huang et al. [16] proposed an end-to-end deep reinforcement learning approach based on graph neural networks for the distributed job shop scheduling problem, demonstrating the application of advanced technologies in solving this complex optimization challenge. D3QN, an abbreviation for dueling double deep Q-network, represents an advancement over DQN. It incorporates the advantages of the dueling DQN and double DQN algorithms. The fundamental premise of D3QN is to address the issue of overestimation of Q -values during training in DQN, with the objective of enhancing the algorithm's overall performance and stability. In light of the considerable state space inherent to DHFJS, the deployment of D3QN proves instrumental in enabling the learning and selection of optimal operators.

In consideration of the existing research, this article proposes the D3QN-based dual-population NSGA-II (D3QN-NSGA-II) algorithm as a potential solution to the energy-efficient scheduling optimization problem of DHFJS. The objective is to identify the solution that minimizes both the maximum completion time and total energy consumption. Compared with existing studies, the main contributions of this algorithm are: 1) designing a two-population evolutionary strategy to enhance the

global and local optimization capabilities by differentially assigning search targets through a reproductive capacity evaluation mechanism; 2) constructing an operator dynamic selection framework that incorporates deep reinforcement learning, and adaptively matching the optimal strategy from 12 local search operators by using D3QN; and 3) introducing an energy-efficient scheduling strategy to achieve energy-efficiency and energy-saving optimization by optimizing the machine idle time to achieve the synergistic optimization of energy efficiency and efficiency. The superiority of the D3QN-NSGA-II in solving the multiobjective heterogeneous flexible job shop scheduling problem is validated through simulation experiments on case studies, which demonstrate its capacity to obtain superior Pareto solutions.

The remainder of this article is structured as follows. Section II formulates the DHFJS problem, defines relevant symbols, and presents the objective function with constraints. Section III details the proposed D3QN-NSGA-II algorithm, including encoding strategies, dual-population evolution, local search operators, energy-saving strategies, and D3QN-based operator selection. Section IV describes the experimental setup, parameter validation, component separation experiments, and comparative results. Finally, Section V concludes the article and outlines future research directions.

II. PROBLEM STATEMENT AND METHOD FRAMEWORK

A. Problem Description

In the scheduling of distributed heterogeneous flexible job shops, there are several jobs $I = \{1, 2, \dots, i, \dots, n\}$, each job consists of multiple operations $J_i = \{O_{i,1}, O_{i,2}, \dots, O_{i,j}, \dots, O_{i,n_i}\}$, which are assigned to various factories $F = \{1, 2, \dots, f, \dots, n_f\}$, each factory contains a set of $M = \{1, 2, \dots, k, \dots, m\}$. Each operation J_i can be executed by a subset of machines from the machine set M in factory F . Each machine is capable of processing multiple operations from different jobs. Once an operation is assigned to a specific factory, the machines within that factory will complete all processing tasks for that operation. The processing time $T_{i,j,f,k}$ varies on different machines within the same factory, depending on the job i , operation j , factory f , and machine k .

Furthermore, the following constraints must be observed during the processing phase.

- 1) Each job can be allocated to a single factory, and all operations must be executed within that factory.
- 2) The set of available machines for each operation is consistent across factories, yet the processing times may vary between factories.
- 3) Each operation of a job can be processed by a single machine at any given time.
- 4) Once an operation has commenced, it must be completed before another operation may be initiated.
- 5) Jobs are of equal priority.
- 6) No precedence constraints exist between operations of different jobs; however, precedence constraints are observed among operations of the same job.

- 7) All jobs and machines are permitted to commence processing at time zero.

B. Symbol Definition

Sets:

I : Set of jobs $I = \{1, 2, \dots, i, \dots, n\}$

J_i : Set of operations for each Job $J_i = \{O_{i,1}, O_{i,2}, \dots, O_{i,j}, \dots, O_{i,n_i}\}$

F : Set of factories $F = \{1, 2, \dots, f, \dots, n_f\}$

M : Set of machines $M = \{1, 2 \dots k \dots m\}$

$K_{f,i,j}$: Set of optional machines for operation $O_{i,j}$ in factory f

$P_{f,k}$: Set of positions where machine k operates in factory f

$T_{i,j,f,k}$: Processing time of operation $O_{i,j}$ on machine k in factory

Parameters:

i : Job index

j : Operation index

k : Machine index

f : Factory index

n : Total number of jobs

m : Total number of machines

n_f : Total number of factories

n_i : Number of operations for each job

E_W : Total energy consumption during working time

E_I : Total energy consumption during idle time

W_P : Machine processing power

W_s : Machine idle power

Variables:

$S_{f,i,j}$: Start time of operation $O_{i,j}$ for job i in factory f

$C_{f,i,j}$: End time of operation $O_{i,j}$ for job i in factory f

$B_{f,k,t}$: Start time of machine k in factory f at position t

$X_{i,j,f,k,t}$: A binary variable indicating whether operation $O_{i,j}$ is processed on machine k at position t in factory f : 1 for processing, 0 otherwise.

$Y_{i,f}$: A binary variable indicating whether job i is assigned to factory f for processing: 1 if assigned, 0 otherwise.

C. Objective Function

In this article, DHFJS's objective function comprises minimizing the maximum completion time $C_{\max}$ and total energy consumption TEC as two objectives, and its modeling follows the manufacturing energy assessment standards and production scheduling specifications:

$$\min \{C_{\max}, \text{TEC}\} \quad (1)$$

- 1) Minimization of the maximum completion time $C_{\max}$

$$\min C_{\max} = \max \{C_{f,i,n_i}\} (\forall i \in I, f \in F). \quad (2)$$

- 2) Total energy consumption TEC

$$\text{TEC} = E_W + E_I \quad (3)$$

$$E_W = \sum_{i \in I} \sum_{j \in J_i} \sum_{f \in F} \sum_{k \in M} \sum_{t \in P_k} W_P T_{i,j,f,k} X_{i,j,f,k,t} \quad (4)$$

$$E_I = \sum_{f \in F} \sum_{k \in M} \sum_{t \in P'_k} W_s \left(B_{f,k,t+1} - B_{f,k,t} - \sum_{i \in I} \sum_{j \in J_i} X_{i,j,f,k,t} T_{i,j,f,k} \right). \quad (5)$$

The constraints are as follows:

$$\sum_{f \in F} Y_{i,f} = 1 (\forall i \in I) \quad (6)$$

$$\sum_{k \in K_{f,i,j}} \sum_{t \in P_k} X_{i,j,f,k,t} = Y_{i,f} (\forall i \in I, j \in J_i, f \in F) \quad (7)$$

$$S_{f,i,n_i} + \sum_{k \in K_{f,i,j}} \sum_{t \in P_k} T_{i,n_i,f,k} X_{i,n_i,f,k,t} \leq C_{\max} (\forall i \in I, f \in F) \quad (8)$$

$$C_{f,\max} \leq C_{\max} (\forall f \in F) \quad (9)$$

$$S_{f,i,j} + \sum_{k \in M} \sum_{t \in P} T_{i,j,f,k} X_{i,n_i,f,k,t} \leq S_{f,i,j+1} (i \in I, j \in J_i - 1, f \in F) \quad (10)$$

$$\sum_{i \in I} \sum_{j \in J_i} X_{i,j,f,k,t} \leq 1 (\forall f \in F, k \in M, t \in P_{f,k}) \quad (11)$$

$$\sum_{i \in I} \sum_{j \in J_i} X_{i,j,f,k,t} \geq \sum_{i \in I} \sum_{j \in J_i} X_{i,j,f,k,t+1}, (\forall f \in F, k \in M, t \in P'_{f,k}) \quad (12)$$

$$B_{f,k,t+1} - B_{f,k,t} \geq \sum_{i \in I} \sum_{j \in J_i} X_{i,j,f,k,t} T_{i,j,f,k} (\forall f \in F, k \in M, t \in P'_{f,k}) \quad (13)$$

$$E_{I,f,k,t} \geq (B_{f,k,t+1} - B_{f,k,t} - \sum_{i \in I} \sum_{j \in J_i} X_{i,j,f,k,t} T_{i,j,f,k}) W_s (\forall f \in F, k \in M, t \in P'_{f,k}) \quad (14)$$

$$B_{f,k,t} \geq S_{f,i,j} - L(1 - X_{i,j,f,k,t}) (\forall i \in I, j \in J_i, f \in F, k \in M, t \in P_{f,k}) \quad (15)$$

$$B_{f,k,t} \leq S_{f,i,j} - L(1 - X_{i,j,f,k,t}) (\forall i \in I, j \in J_i, f \in F, k \in M, t \in P_{f,k}) \quad (16)$$

$$0 \leq B_{f,k,t} \leq L (\forall i \in I, j \in J_i, f \in F, k \in M, t \in P_{f,k}) \quad (17)$$

$$0 \leq S_{f,i,j} \leq L (\forall i \in I, j \in J_i, f \in F, k \in M, t \in P_{f,k}). \quad (18)$$

Constraint (6) guarantees that each job is assigned exclusively to a single factory for processing. Constraint (7) ensures that once a job has been allocated to a factory, all operations associated with that job will be completed within that factory. Constraint (8) defines the optimization objective, establishing a link between the maximum completion time and the completion times of individual operations. Constraint (9) establishes a connection between the maximum completion times across factories and the overall maximum completion time. Constraint (10) ensures the same workpiece process order constraint through the recursive relationship of process start–end time, and combined with the machine single process constraint of Constraint (11), this set of constraints can be proved to be noncontradictory in the integer programming model by using the topological sorting algorithm. Constraint (12) establishes the condition that the commencement of a subsequent operation on a machine

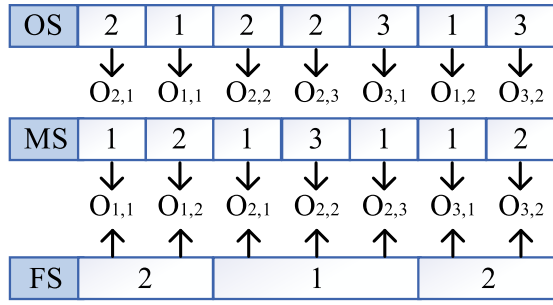


Fig. 1. Coding structure.

is contingent upon the conclusion of the preceding operation. Constraint (13) provides a comprehensive description of the relationship between the commencement and conclusion times of operations on each machine. Constraint (14) is employed for the calculation of idle energy consumption of machines. Constraints (15) and (16) relax equality constraints in order to enhance the MILP model's applicability for the branch-and-bound algorithm. This is accomplished by establishing that the commencement time of a machine operation is equivalent to the operation's own commencement time. Constraints (17) and (18) delineate the permissible range of decision variables.

III. D3QN-NSGA-II ALGORITHM

A. Encoding and Population Initialization

The encoding pattern in this study employs a three-tiered approach, as illustrated in Fig. 1, which represents the encoding method for a solution. In this context, OS denotes operation selection, MS denotes machine selection for each operation, and FS denotes the sequence of factories assigned to each job. The length of the OS and MS vectors is proportional to the total number of operations, whereas the FS vector aligns with the total number of jobs. In the decoding phase, the FS vector is utilized to assign each job to the corresponding heterogeneous factories. Subsequently, in accordance with the MS vector, each operation selects an optimal machine from the available options, thereby facilitating the processing of $O_{i,j}$. Finally, this yields the start and completion times for each operation, thereby determining the overall job shop makespan and TEC. For detailed encoding and decoding mechanisms, please refer to the literature [17].

In the context of D3QN training, the use of random initialization is a key strategy for achieving enhanced diversity. This entails randomly ordering the sequences of jobs and operations, thereby generating an ordering code, OS. Subsequently, within the operation code, the machines that are capable of processing each operation are selected from their respective sets, thereby creating the machine sequence MS. Subsequently, operations are randomly assigned to processing factories based on the operation code, thereby generating the factory sequence, FS.

B. Dual Population Evolutionary Strategy

In the domain of multiobjective optimization, conventional evolutionary algorithms typically adhere to a single

Algorithm 1: Reproductive Capacity Assessment Mechanism.

```

1: Input: Chromosome population size  $N$ , test
   population size  $S$ , population division ratio parameter
    $\beta$ 
2: Output: Reproductive subpopulation pop1, ordinary
   subpopulation pop2
3: Generate a test population of size  $S$  randomly
4: For  $i=1$  To  $N$ 
5:   For  $j=1$  to  $S$  do
6:     a) Perform crossover between individual
        $i$  and individual  $j$  to produce offspring
7:     b) If the offspring dominates individual  $i$ ,
       replace individual  $i$  with the offspring and
       increment the reproductive capability of
       individual  $i$  by 1
8:   End
9: End
10: For  $i=1$  To  $N$  do
11:   If(reproductive capability of individual  $i$  > average
     reproductive capability)
12:     Individual  $i$  belongs to the reproductive
     subpopulation pop1
13:   Else
14:     Individual  $i$  belongs to the ordinary
     subpopulation pop2
15: End for

```

evolutionary trajectory, which can impede their capacity to explore extensively and accelerate convergence, thereby increasing outcome uncertainty. To surmount these obstacles, the incorporation of a dual-population evolutionary methodology seeks to augment the directional search capabilities and environmental adaptability of the NSGA-II algorithm. This expansion broadens the search horizon, thereby reducing the probability of becoming trapped in a local optimum. Throughout the iterative course of the algorithm, the primary population is divided into two subsets, each undergoing distinct genetic strategies, guided by designated population ratio parameters and a sex-based partitioning mechanism [18]. In the context of multiobjective optimization, the procedural specifics of population partitioning via the sex-based mechanism are delineated by Algorithm 1.

To form subpopulation 1, which is composed of $N \times \beta$ individuals with robust reproductive capabilities, a selection process is first employed to identify the individuals who will comprise this subpopulation. In the event that the requisite number is not available, individuals exhibiting strong reproductive abilities from the general population may be included as a supplement. The remaining individuals constitute subpopulation 2. Individuals in subpopulation 1, due to their robust reproductive capabilities, undergo standard POX crossover and random two-point exchange mutation for process sequence ordering. The machine selection process employs UX crossover and random machine replacement mutation, whereas the factory

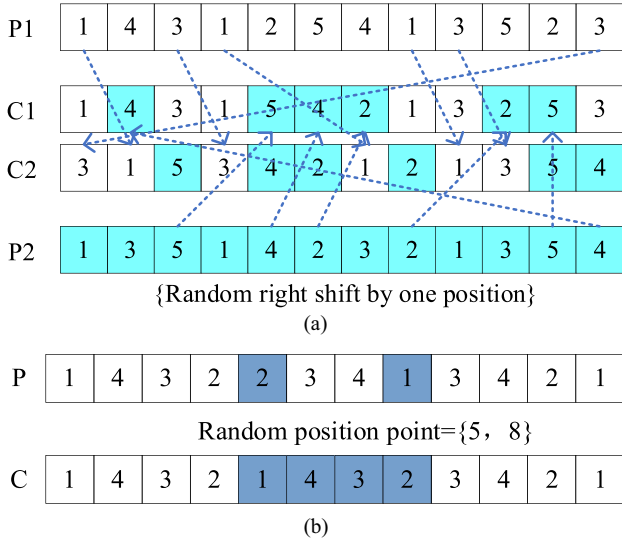


Fig. 2. Population 2 process selection, crossover, and mutation operations. (a) Improve POX crossover. (b) Reverse mutation.

allocation method utilizes standard UX crossover to maintain a global population advantage. In the case of subpopulation 2, the use of traditional genetic algorithms is unable to generate new individuals in a satisfactory manner, which in turn limits the efficiency of local search. Therefore, we employ an improved POX crossover method and genetic sequence inversion mutation strategy for process sequence ordering [as shown in Fig. 2(a) and (b)]. Machine selection utilizes uniform crossover and mutation by randomly selecting machines with shorter processing times to enhance algorithm convergence.

C. Local Search Operators

In addressing the energy-efficient scheduling problem of DHFJS, a number of efficient and widely accepted local search strategies have been employed, including positional exchange, posterior insertion, and anterior insertion. During the search process, the application of operators related to critical paths has the potential to significantly enhance the quality of solutions generated in each iteration. When the algorithm encounters a local optimum, the utilization of disruptive reconstruction operators markedly increases the probability of the algorithm escaping from local optima.

This article introduces three categories of local search operators, each with distinct properties. The first category comprises classical operators, the second includes disruptive and reconstructive operators, and the third is comprised of critical path operators. These categories and their respective properties are outlined in Table I.

D. Energy Saving Strategy

In the context of scheduling problems with consideration of energy consumption, the total energy consumption is a critical metric that significantly impacts the achievement of green scheduling. One principal method for reducing energy consumption during the production of jobs is to minimize both

machine processing time and idle time. In accordance with this principle, an energy-saving strategy has been devised which places an emphasis on local optimization and the modification of production sequences with a view to reducing idle time on machines and, consequently, lowering overall energy consumption throughout the production process.

The strategy is based on the following principles: 1) *local optimization*: the application of local search techniques is utilized for the optimization of operational sequencing on machinery, with the objective of minimizing downtime and enhancing machine utilization; 2) *adjusting production sequences*: the re-ordering of operations or the adjustment of their start times may be employed to more effectively utilize machine capacity and reduce periods of inactivity; and 3) *reduction of idle time*: the objective is to identify and reduce instances of idle time between operations, thereby reducing unnecessary energy consumption.

As shown in Fig. 3, operation O_{31} on machine $M2$ must wait until operation O_{22} is completed before commencing, while operation O_{32} on $M1$ must wait for operation O_{23} to finish before starting, resulting in significant idle time. Under the premise of maintaining scheduling constraints, shifting operation O_{31} on $M2$ before O_{22} and moving operation O_{32} on $M1$ before O_{23} can reduce machine idle time, thereby lowering idle energy consumption and shortening the completion time of jobs.

This strategy, which entails maintaining the sequence of operations while rearranging the production order of different jobs, has the effect of minimizing machine idle periods. Without modifying the critical path, it contributes to a reduction in overall energy consumption and an enhancement in production efficiency.

E. Operator Selection Based on D3QN

The 12 local search operators in Table I are each suited to different optimization scenarios and optimal solutions. In the local search phase of population evolution, when optimizing the Pareto front solution, the optimal strategy needs to be dynamically selected from 12 operators. To this end, this section proposes a D3QN-based operator selection mechanism, which is invoked in the nondominated solution optimization phase of each iteration (Algorithm 3, line 25), and achieves adaptive matching of operators through deep reinforcement learning.

The DQN algorithm [19] is an efficient unsupervised deep reinforcement learning algorithm capable of autonomously learning the data distribution within the solution space. In light of these distribution characteristics, a probability mapping related to operator selection is constructed. The original DQN algorithm is prone to overestimating Q -values in certain cases, which can result in instability and a decline in performance during the learning process. To address this issue, an improved version, known as double DQN (DDQN), was proposed. DDQN introduces two neural networks: one for selecting actions and computing target Q -values (referred to as the prediction network), and another for evaluating target Q -values (known as the target network). During the training phase, the target network is periodically updated to generate target Q -values in lieu of utilizing Q -values directly from the prediction network. This

TABLE I
HEURISTIC OPERATOR DESIGN

Operator Types	Operator Designs
Classical operators	Randomly select two operations, swap their positions.
	Randomly select one operation and insert it at a specified position.
	Randomly select two operations, insert one operation before another.
	Randomly select two operations, insert one operation after another.
Disruptive and reconstructive operators	Randomly generate a binary array (0–1), retain positions with 1, randomly redistribute positions with 0.
	Randomly select a point, maintain positions to the right, shuffle positions to the left.
	Randomly select a point, maintain positions to the left, shuffle positions to the right.
	Randomly select two points, disrupt operations between the two points.
Critical path operators	Randomly select an operation on the critical path, randomly move all operations of its job to another factory.
	Randomly select an operation on the critical path, use a roulette wheel selection method to choose a processing factory, move all operations of its job to the selected factory.
	Randomly select an operation on the critical path, randomly reselect a machining center within its available range.
	Randomly select an operation on the critical path, use a roulette wheel selection method to choose a machining center, move the current operation to the selected machine for processing.

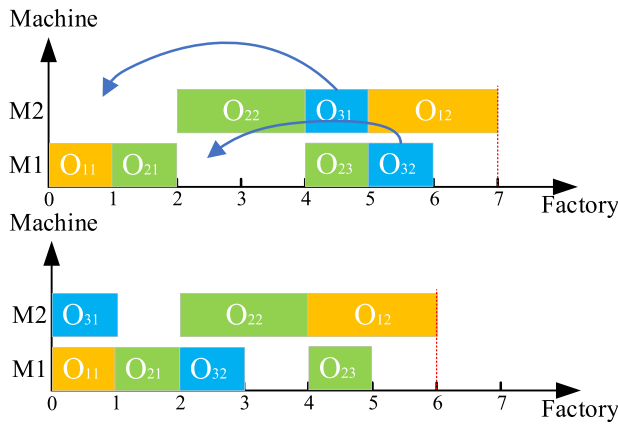


Fig. 3. Energy saving strategies.

Algorithm 2: Reward Mechanism.

```

1: Input:  $C'_{max}$ ,  $TEC'$ ,  $C_{max}$ ,  $TEC$ 
2: Output: reward
3: If  $C_{max} < C'_{max}$  and  $TEC < TEC'$  :
4:   reward = 5
5: Else if  $C_{max} < C'_{max}$ ,  $TEC > TEC'$  or  $C_{max} >$ 
    $C'_{max}$ ,  $TEC < TEC'$ :
6:   reward = 10
7: Else:
8:   reward = 0
9: End

```

approach mitigates the potential for overestimation. Equation (19) defines the objective function for DQN, while (20) defines the objective function for DDQN

$$Y_t^{DQN} = R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \theta_t^-) \quad (19)$$

$$Y_t^{DoubleDQN} = R_{t+1} + \gamma \max_a Q(S_{t+1}, \arg\max_a Q(S_{t+1}, a; \theta_t); \theta_t^-) \quad (20)$$

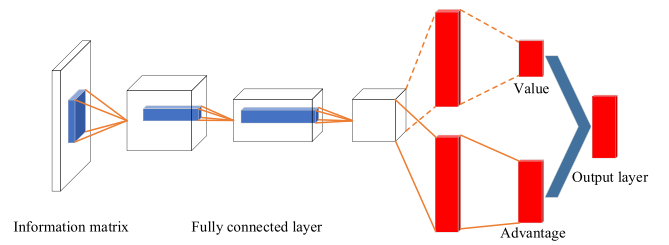


Fig. 4. Network architecture of D3QN.

The Q -value, represented by $Q(s, a)$, is an implicit indication of the value associated with being in state s and the advantage gained from selecting action a . In response to this, the Dueling network [20] was developed with the explicit objective of segregating the state value and action advantage components within a given state. In the context of the dueling network framework, two distinct networks are employed with the objective of estimating the state value function, $V(s)$, and the action advantage function, $A(s, a)$, for each state. Subsequently, the action value function $Q(s, a)$ is derived by integrating the aforementioned networks, as illustrated in the following equation:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|A|} \sum_{a'} A(s, a'; \theta, \alpha) \right) \quad (21)$$

Fig. 4 illustrates the network architecture of the D3QN, depicting how the state inputs are processed through the hidden layers to generate action values, highlighting the separation of state value and action advantage branches in the dueling network design. The D3QN network consists of an input layer, three fully connected layers (with 64, 128, and 64 neurons, respectively, all using the ReLU activation function), a branching fully connected layer (split into the value stream and advantage stream), and an output layer.

State design: The local search operators are recommended based on the current job shop state, making accurate extraction and representation of this state crucial. Considering the study's

Algorithm 3: D3QN-NSGA-II Algorithm Framework.

Input: Number of iterations (G), population size (ps), scaling parameter (β), crossover probability (P_c), mutation probability (P_m), batch size (batch), learning rate (α), epsilon-greedy coefficient (ϵ), discount factor (γ)

Output: Non-dominated solution set PF

- 1: Initialize population randomly
- 2: Initialize D3QN network ($batch, \alpha, \epsilon, \gamma$)
- 3: **For** $g=1$ to G **do**
- 4: $pop1, pop2 \leftarrow$ Reproductive capacity assessment mechanism split population
- 5: $pop1, pop2 \leftarrow$ Binary tournament selection ($pop1, pop2$)
- 6: **For** $i=1$ to $\text{len}(pop1)$ **do**
- 7: $C1, C2 \leftarrow$ Standard evolution operators (P_c, P_m)
- 8: $pop1 \leftarrow pop1 \cup C1 \cup C2$
- 9: $pop1 \leftarrow$ Remove duplicates ($pop1$)
- 10: $pop1 \leftarrow$ Energy-saving strategy ($pop1$)
- 11: **End**
- 12: **For** $i=1$ to $\text{len}(pop2)$ **do**
- 13: $C1, C2 \leftarrow$ Improved evolution operators (P_c, P_m)
- 14: $pop2 \leftarrow pop2 \cup C1 \cup C2$
- 15: $pop2 \leftarrow$ Remove duplicates ($pop2$)
- 16: $pop2 \leftarrow$ Energy-saving strategy ($pop2$)
- 17: **End**
- 18: $C \leftarrow$ Remove duplicates ($pop1 \cup pop2$)
- 19: $\{F0, F1, \dots\} \leftarrow$ Fast non-dominated sorting (C)
- 20: $C' \leftarrow$ Crowding distance calculation ($\{F0, F1, \dots\}$)
- 21: $pop \leftarrow C' (1:ps, :)$
- 22: $P \leftarrow$ Remove duplicates ($PF \cup F0$)
- 23: $P, Q_\theta \leftarrow$ Operator selection mechanism based on D3QN ($P, batch, \alpha, \epsilon, \gamma$)
- 24: $g = g + 1$
- 25: **End**
- 26: $PF \leftarrow$ Fast non-dominated sorting (P)

constraints related to time and energy consumption, three metrics relevant to the overall scheduling problem (number of jobs, number of machines, total number of operations), four metrics related to job shop processing time (total machine completion time, machine idle/processing time, total completion time per factory, machine idle/processing time per factory), and two metrics related to energy consumption (total machine energy consumption, energy consumption per factory) are proposed to accurately characterize the current job shop state.

Action design: Relevant actions involve selecting one of the 12 local search operators as shown in Table I.

Reward design: The designed reward is determined based on the current state $C_{\max}$, TEC, and the previous state $C'_{\max}$, TEC' as outlined in Algorithm 2.

F. D3QN-NSGA-II Algorithm Flow

Sections II-A to II-E provided a detailed introduction to the main components of the D3QN-NSGA-II algorithm. Initially, the algorithm initializes parameters including population size, scaling parameter β , crossover probability P_c , mutation probability P_m , and sets up the D3QN network. Subsequently, using the NSGA-II algorithm, the population undergoes rapid nondominated sorting and crowding distance calculation. It

employs a reproductive capacity assessment mechanism to split the population based on the scaling parameter β , and through a dual-population evolution strategy and energy-saving scheduling strategy, it obtains a new generation of the population. The nondominated solutions from the latest generation are stored in PF. Each member of the PF population undergoes selection of a local search operator, and the resultant solution from executing this operator is compared against the original solution to ascertain its reward. During each decision-making round, the current state S_t , action chosen under this state A_t , obtained reward R_t , and the next state S_{t+1} are combined to form experiences stored in the experience pool. If the experience pool reaches a predefined threshold, the DQN network is trained. The framework of the D3QN-NSGA-II algorithm is illustrated in Algorithm 3.

IV. EXPERIMENTAL SETTING AND RESULTS

A. Experimental Parameters

In order to validate the effectiveness of D3QN-NSGA-II, this section employs the use of 20 test cases, as provided in the relevant literature [21]. These cases cover ranges for the number of jobs $n \in \{10, 20, 30, 40, 50, 100, 150, 200\}$ and the number of factories $n_f \in \{2, 3, 4, 5, 6, 7\}$. Each factory is

TABLE II
PARAMETER VALIDATION RESULTS FOR EACH ALGORITHM

Algorithm	Parameter	Value Range	Optimal Value
D3QN-NSGA-II	Population size ps	80, 90, 100	100
	Crossover probability P_c	0.8, 0.9, 1	0.9
	Mutation probability P_m	0.1, 0.2, 0.3	0.2
	Learning rate α	0.001, 0.01, 0.1	0.001
	Greedy factor ϵ	0.7, 0.8, 0.9	0.9
	Discount factor γ	0.7, 0.8, 0.9	0.9
	Experience pool	512, 1024	512
	Batch size bs	8, 16, 24	16
MOEA/D	Population size ps	100, 200, 300	200
	Neighborhood size ns	10, 15, 20	15
	Crossover probability P_c	0.8, 0.9, 1	1
	Mutation probability P_m	0.1, 0.2, 0.3	0.2
MOPSO	Population size ps	50, 100, 150	100
	Learning factors c_1, c_2	1.5, 2, 2.5	2
	Inertia weight coefficient ω	0.7, 0.8, 0.9	0.7
NSWOA	Population size ps	100, 200, 300	200
	Parameter a	$a \in [0, 2]$	Linear decrement
	Contraction-expansion coefficient A	$A \in [-1, 1]$	Random generation
	Proximity coefficient C	$C \in [0, 2]$	Random generation
MOGWO	Population size ps	100, 200, 300	100
	Parameter a	$a \in [0, 2]$	Linear decrement
	Random exploration coefficients A, C	$A \in [-2, 2], C \in [0, 2]$	Random generation

TABLE III
FRIEDMAN RANK SUM TEST FOR
D3QN-NSGA-II VARIANT ALGORITHMS

Variant Algorithms	HV Rank	IGD Rank
NSGA-II	4.56	4.34
DP+NSGA-II	3.81	3.57
ES+NSGA-II	3.31	3.34
D3QN+NSGA-II	1.95	2.15
D3QN-NSGA-II	1.33	1.6

comprised of five machines, and each job is comprised of five operations. The selection of machines for each operation allows for the consideration of multiple options, with processing times varying according to the specific factory and machine chosen. All machines have a processing power $W_P = 4$ and an idle power $W_s = 1$. To analyze the distribution and convergence of solution sets obtained by the proposed algorithm and comparative algorithms, the hypervolume(HV) metric [22] and the inverted generational distance (IGD) are introduced. A higher HV is indicative of superior overall algorithmic performance, whereas a smaller IGD is indicative of superior convergence and distribution characteristics. Comparative algorithms include classical multiobjective optimization algorithms such as MOEA/D and several state-of-the-art swarm intelligence optimization algorithms like MOPSO [23], NSWOA [24], and MOGWO [25].

B. Parameter Validation

The efficacy of an algorithm is significantly influenced by its key parameters. Prior to conducting simulation experiments, it is of paramount importance to validate and optimize the parameters for both the proposed algorithm and the selected comparative algorithms. To ensure fairness and consistency in the comparison, all algorithms (including MOEA/D, MOPSO,

NSWOA, and MOGWO) adopt the same Taguchi method for parameter tuning, which is used to assess the impact of various parameters on algorithm performance and obtain optimal parameter combinations. The experiments are conducted based on the 20J3F test case, where each parameter combination is independently run ten times, and the results of the parameter validation are presented in Table II. For comparative algorithms, parameters were not directly adopted from original literature but retuned specifically for the DHFJS problem in this study, following the same Taguchi method framework as D3QN-NSGA-II to ensure comparable experimental conditions.

C. Component Separation Experiment

To assess the contribution of each component in the proposed strategy, we designed a comparative experiment to evaluate the effects of the dual-population evolutionary strategy, energy-efficient scheduling strategy, and D3QN-based operator selection mechanism on algorithm performance. The experiment compared the basic NSGA-II algorithm, NSGA-II with only the dual-population evolutionary strategy (DP+NSGA-II), NSGA-II with only the energy-efficient scheduling strategy (ES+NSGA-II), NSGA-II with only the D3QN-based operator selection mechanism (D3QN+NSGA-II), and the complete algorithm (D3QN-NSGA-II). To ensure the reliability of the experimental results, identical parameter settings were used for all the algorithms, and each variant was run ten times across 20 test cases to compute the average performance.

This study employed SPASSAU software to analyze the significance of the relationships between algorithm variants, demonstrating the effectiveness of the proposed improved operators. Table III presents the Friedman rank and test statistics for each variant algorithm. Taking the HV value as an example,

TABLE IV
COMPARATIVE RESULTS OF HV (MAX) METRIC FOR THE ALGORITHMS

Ins	MOEA/D		MOPSO		NSWOA		MOGWO		D3QN-NSGA-II	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std
10J2F	0.4531	0.0316	0.3570	0.0538	0.2021	0.1355	0.0398	0.0129	0.5779	0.1324
20J2F	0.3493	0.0832	0.0153	0.0093	0.4381	0.0375	0.5857	0.2364	0.6587	0.0987
20J3F	0.6579	0.0591	0.7312	0.0371	0.4975	0.0945	0.3465	0.1103	0.7851	0.1589
30J2F	0.4689	0.0695	0.2298	0.1134	0.7226	0.0682	0.1986	0.0569	0.7479	0.1754
30J3F	0.1531	0.0412	0.3894	0.0657	0.0274	0.0016	0.3062	0.1568	0.5874	0.1589
40J2F	0.3174	0.0862	0.0627	0.0068	0.6841	0.2138	0.0478	0.0057	0.2578	0.0789
40J3F	0.4572	0.0737	0.1257	0.0381	0.5857	0.1578	0.2156	0.0089	0.7923	0.2587
40J4F	0.2671	0.1502	0.7646	0.1284	0.6731	0.0874	0.2526	0.1367	0.8751	0.1569
50J3F	0.3892	0.2781	0.7603	0.0429	0.0772	0.0051	0.5621	0.0985	0.7925	0.1175
50J4F	0.1883	0.0145	0.5440	0.1254	0.3863	0.0645	0.5978	0.1236	0.6244	0.0791
50J5F	0.4307	0.1358	0.6589	0.0487	0.7007	0.1654	0.6321	0.2238	0.7015	0.1267
100J4F	0.6566	0.1787	0.6116	0.0503	0.3909	0.1007	0.5784	0.0457	0.7954	0.1598
100J5F	0.0314	0.0144	0.7808	0.1994	0.4189	0.0456	0.3547	0.1032	0.5789	0.0563
100J6F	0.2612	0.0248	0.3951	0.1754	0.7192	0.0954	0.5368	0.0781	0.7965	0.0654
100J7F	0.2176	0.0187	0.3404	0.0871	0.1617	0.0093	0.4568	0.1597	0.5798	0.0068
150J5F	0.4588	0.1632	0.5609	0.1891	0.4578	0.1257	0.1248	0.0548	0.8645	0.2548
150J6F	0.2535	0.0632	0.6831	0.1203	0.5814	0.1268	0.4583	0.1587	0.7128	0.1493
150J7F	0.3445	0.0092	0.5946	0.0891	0.1357	0.0568	0.5462	0.2487	0.6568	0.0574
200J6F	0.1819	0.0227	0.3223	0.636	0.5978	0.1265	0.3784	0.0747	0.7536	0.1228

TABLE V
COMPARATIVE RESULTS OF IGD (MIN) METRIC FOR THE ALGORITHMS

Ins	MOEA/D		MOPSO		NSWOA		MOGWO		D3QN-NSGA-II	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std
10J2F	0.8954	0.0698	0.8453	0.2498	0.6548	0.1587	0.7897	0.2148	0.5647	0.1297
20J2F	0.5659	0.0935	0.7449	0.146	0.4587	0.0817	0.4288	0.1355	0.4034	0.0455
20J3F	0.7991	0.1265	0.3366	0.0721	0.5305	0.1264	0.5654	0.1278	0.2725	0.0528
30J2F	0.3657	0.0196	0.3771	0.1301	0.2997	0.0654	0.5478	0.0984	0.4235	0.1789
30J3F	0.6674	0.2345	0.7801	0.1687	0.6328	0.1056	0.8234	0.2341	0.5632	0.1740
40J2F	0.6025	0.1874	0.4427	0.1309	0.7981	0.1321	0.5244	0.0789	0.4563	0.0896
40J3F	0.4125	0.0805	0.7922	0.2287	0.8745	0.0457	0.4523	0.1365	0.3725	0.1163
40J4F	0.3405	0.1112	0.5357	0.0798	0.3544	0.0847	0.6546	0.2789	0.3145	0.0405
50J3F	0.4501	0.0782	0.6377	0.1588	0.2112	0.0566	0.7156	0.1789	0.4239	0.1087
50J4F	0.5668	0.2176	0.8124	0.2683	0.6486	0.1254	0.5634	0.0784	0.4561	0.2023
50J5F	0.7938	0.0623	0.7114	0.3844	0.6741	0.1923	0.5786	0.2132	0.4896	0.1232
100J4F	0.2975	0.1058	0.6887	0.1848	0.5874	0.0781	0.3789	0.0051	0.3687	0.1327
100J5F	0.8157	0.1964	0.5071	0.0922	0.7213	0.2360	0.4215	0.1235	0.4023	0.1578
100J6F	0.5762	0.1436	0.8019	0.2378	0.4369	0.1298	0.7835	0.2654	0.3207	0.0908
100J7F	0.4164	0.1233	0.3704	0.1142	0.4234	0.0563	0.4878	0.1907	0.4541	0.1147
150J5F	0.7623	0.0987	0.8248	0.2147	0.7894	0.1847	0.6781	0.2556	0.5232	0.1245
150J6F	0.3745	0.1305	0.3974	0.0855	0.2945	0.1004	0.4575	0.1035	0.1124	0.0078
150J7F	0.8437	0.0666	0.2996	0.0271	0.3478	0.0905	0.4023	0.0879	0.2669	0.1308
200J6F	0.3113	0.0161	0.6447	0.1665	0.5448	0.2654	0.4565	0.1407	0.2955	0.0481

Note: Bold indicates that the algorithm achieved the optimal results.

the test statistic is calculated using the below equation

$$\chi^2 = \frac{12b}{k(k+1)} \sum_{i=1}^k \bar{R}_i^2 - 3b(k+1) \quad (22)$$

which yields a value of 57.1488. Referring to the chi-square distribution table, with a degree of freedom of 4, a p -value significantly smaller than 0.001 indicates that there are significant differences between the five algorithm variants. The experimental results show that the complete algorithm (D3QN-NSGA-II) outperforms other variants in both HV and IGD metrics. Specifically, the dual-population evolutionary strategy and D3QN-based operator selection mechanism contributed the most to the performance improvements. In particular, the dual-population strategy effectively enhanced the population diversity and global search capability, while the D3QN operator

selection mechanism significantly improved local search efficiency. The energy-efficient scheduling strategy also played a crucial role in reducing energy consumption, further enhancing the overall performance of the algorithm.

D. Comparative Experiments

To provide a comprehensive assessment of the proposed algorithm's overall performance, it was benchmarked against three swarm intelligence algorithms and a classical multiobjective optimization method. The benchmarking process included the multiobjective evolutionary algorithm based on decomposition (MOEA/D), enhanced multiobjective particle swarm optimization (MOPSO), enhanced nondominated sorting whale optimization algorithm (NSWOA), and enhanced multiobjective grey wolf optimization algorithm (MOGWO).

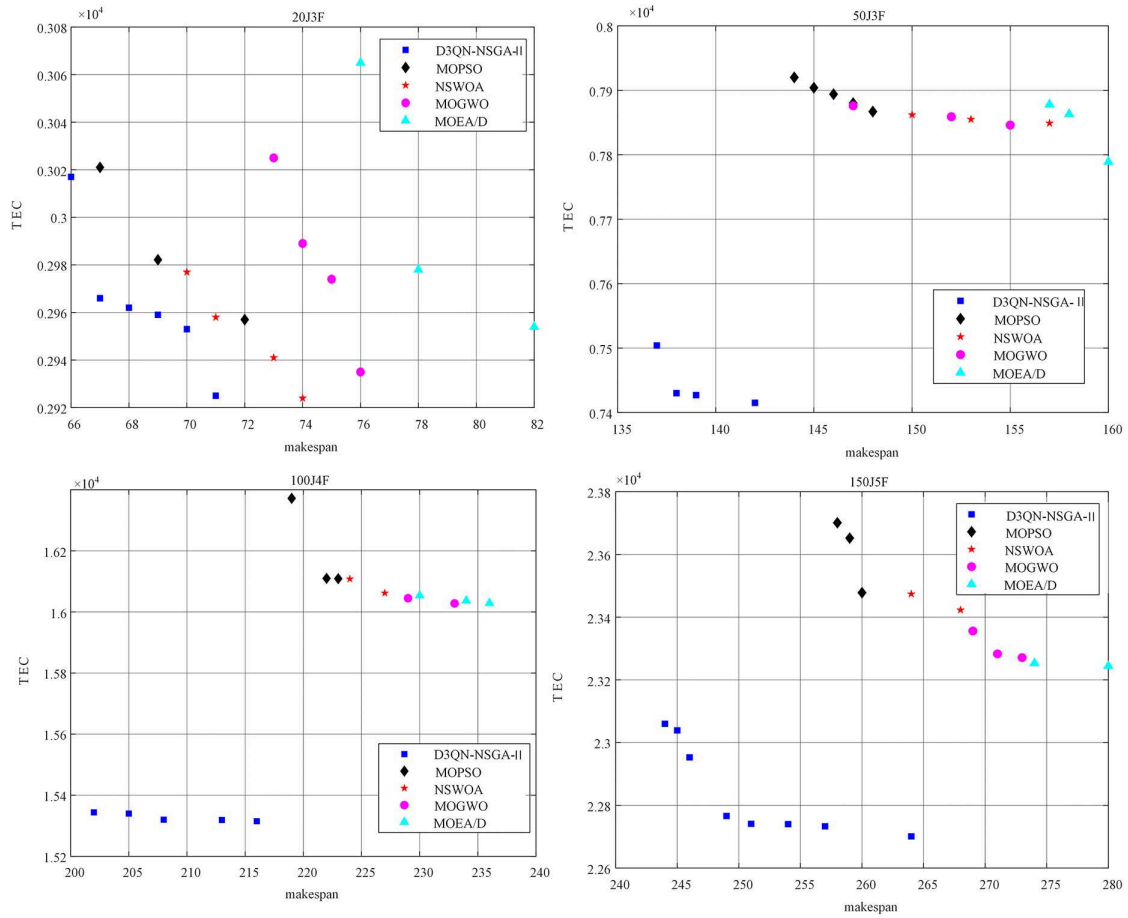


Fig. 5. Distribution of solutions obtained by comparative algorithms on selected instances.

The experiments were conducted on a system equipped with an AMD Ryzen 7 5800H processor with Radeon Graphics, operating at 3.20 GHz, and 16 GB of memory, running a 64-bit Windows 11 Home edition operating system. The algorithms were implemented in Python 3.8 using the Pytorch framework and executed in Pycharm 2024. To ensure the integrity of the comparisons, each algorithm was executed ten times independently for each instance. The convergence and diversity of the proposed algorithms were evaluated by calculating the mean value of the results obtained across the ten runs, which was then used as the basis for the performance assessment.

Tables IV and V present the mean and standard deviation of HV and IGD values for the D3QN-NSGA-II algorithm and four comparison algorithms applied to the DHFJS problem. From these tables, it is evident that D3QN-NSGA-II achieves the highest HV values and the lowest IGD values in the majority of test cases, indicating its significant advantages in both solution convergence and diversity. For instance, in test cases such as 20J3F, 40J4F, and 150J5F, the mean HV values for D3QN-NSGA-II are 0.7851, 0.8751, and 0.8645, respectively, significantly outperforming the other algorithms. The mean IGD values are also as low as 0.2725, 0.3145, and 0.5232, showing a solution set that is much closer to the true Pareto front. To further validate the statistical significance of the improvement,

the Wilcoxon signed-rank test was applied to the HV metric of D3QN-NSGA-II and each of the comparison algorithms, performing pairwise significance analysis. Taking the comparison between D3QN-NSGA-II and MOPSO on the HV metric as an example, the positive rank sum was calculated as $T^+ = 189.0$, and the absolute value of the negative rank sum was $T^- = 1.0$. The test statistic is $\min(T^+, T^-) = 1.0$, and the p -value ($0.0002 < \alpha (0.05)$), indicating a significant difference between the two methods. The experimental results demonstrate that the improvements in both convergence and diversity are statistically significant, validating the effectiveness of the proposed strategy.

Fig. 5 presents the solution distribution of D3QN-NSGA-II compared to other algorithms on four test cases: 20J3F, 50J3F, 100J4F, and 150J5F. The Pareto front of D3QN-NSGA-II performs excellently in both the low $C_{\max}$ and low TEC regions, with a uniformly distributed solution set that can meet the requirements of different optimization priorities. In contrast, the Pareto fronts of MOEA/D and MOPSO are more dispersed, while NSWOA and MOGWO tend to focus on optimizing a single objective in certain instances, sacrificing overall balance. The consistent performance of D3QN-NSGA-II across instances of different scales further validates its robustness and scalability, laying the foundation for deployment in complex

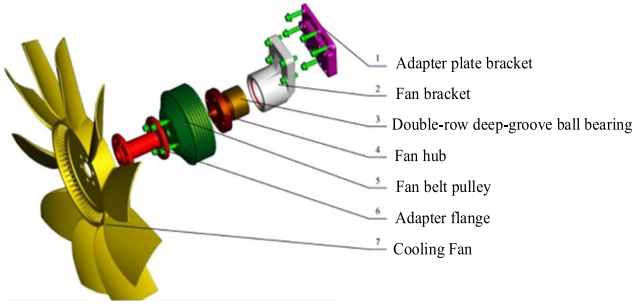


Fig. 6. Engine cooling system.

industrial applications. While D3QN-NSGA-II may have higher computational complexity, its superior solution quality in practical applications outweighs this potential limitation.

Compared to the other algorithms, D3QN-NSGA-II achieves remarkable success in solving the DHFJS problem, owing to its unique design and optimization strategies. First, the dual-population strategy divides the population into reproduction-capable and regular subpopulations, each subjected to differentiated genetic operations. This effectively balances global exploration and local exploitation, broadening the search space and reducing the risk of falling into local optima. Second, the D3QN operator selection mechanism leverages deep reinforcement learning to dynamically choose the optimal local search operator, significantly enhancing the algorithms adaptability to complex state spaces and improving optimization efficiency. Furthermore, the energy-saving strategy reduces total energy consumption by optimizing machine idle times, indirectly shortening the makespan. In addition, all algorithms were run ten times under the same hardware environment, and the results were averaged to ensure stability. The significance test, with a p -value less than 0.05, further confirms the effectiveness of the improvements in D3QN-NSGA-II.

E. Engineering Case Study

This section presents an analysis of a real production order for an engine cooling system from a domestic automotive manufacturing company, using it as an engineering case study. The theories and methods introduced in this chapter are applied to the production scheduling of this practical case. The engine factory responsible for producing this cooling system operates multiple job shops to meet order demands, and production is processed based on the order volume. This issue can be abstracted as a distributed heterogeneous job shop problem. Fig. 6 illustrates the schematic diagram of the engine cooling system. The system consists of components such as the adapter plate bracket, fan bracket, double-row deep groove ball bearings, fan hub, fan belt pulley, adapter flange, and cooling fan. The engine cooling system is produced in two job shops within the factory. Job shop 1 manufactures four parts: adapter plate bracket, fan bracket, fan hub, and adapter flange, while job shop 2 handles the production of the remaining components.

Each job shop is equipped with eight machines for processing these components, including two milling machines (M1, M2),

TABLE VI
RESULTS OF SOLVING THE ENGINEERING CASE USING
DIFFERENT ALGORITHMS

Algorithm	HV		IGD	
	Mean	Std	Mean	Std
D3QN-NSGA-II	0.7951	0.2312	0.1024	0.0509
MOEA/D	0.6831	0.1125	0.2296	0.1081
MOPSO	0.6724	0.2564	0.1260	0.0832
NSWOA	0.7128	0.1943	0.2050	0.1178
MOGWO	0.7732	0.2556	0.1246	0.0852

Note: Bold indicates that the algorithm achieved the optimal results.

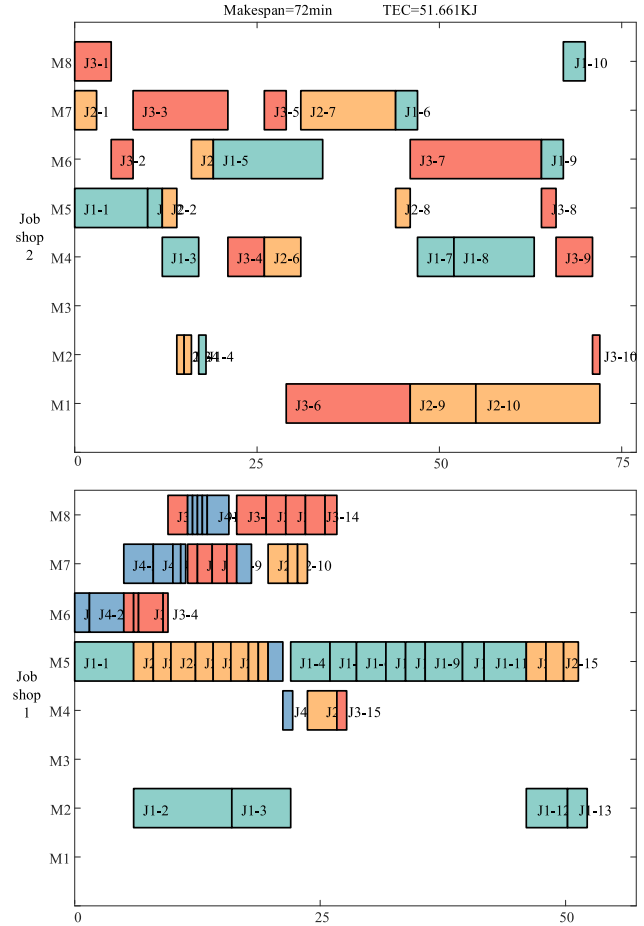


Fig. 7. Gantt chart for the engineering case scheduling.

two drilling machines (M3, M4), one machining center (M5), one manual lathe (M6), and two computer numerical control lathes (M7, M8). Using the same experimental environment and algorithm parameter settings as in Part D, D3QN-NSGA-II was applied alongside methods such as MOEA/D, MOPSO, NSWOA5, and MOGWO to solve this production case. Each algorithm was run ten times to calculate the average results. The solution outcomes for the engineering case are shown in Table VI.

From table, it can be observed that when D3QN-NSGA-II is applied to the engineering case, the results for the HV and IGD metrics outperform those of the other algorithms, indicating that the proposed algorithm demonstrates superior performance in

practical applications. The Gantt chart for the scheduling results obtained by D3QN-NSGA-II for this engineering case is shown in Fig. 7, with a maximum completion time of 72 min and a total energy consumption of 51.661 KJ.

V. CONCLUSION

The D3QN-NSGA-II algorithm incorporates the attributes of deep reinforcement learning and multiobjective optimization. By employing a dual-population strategy and a D3QN-based operator selection mechanism, the algorithm effectively balances global exploration and local exploitation, thereby enhancing both the convergence speed and the quality of solutions. A detailed description and modeling of the DHFJS problem have led to the definition of rational objective functions and constraints, thereby providing a clear framework for algorithm design and application. Comparative experiments with existing algorithms have consistently demonstrated that D3QN-NSGA-II exhibits superior performance across a range of test cases, as evidenced by metrics such as HV and IGD. These findings validate the effectiveness and superiority of D3QN-NSGA-II.

From a management practice perspective, this study provides a scheduling method for the manufacturing industry that optimizes both efficiency and energy consumption. The dual-population evolutionary strategy guides companies in dynamically allocating search resources during production scheduling, improving global optimization capabilities. The D3QN-based operator selection mechanism helps adaptively adjust scheduling strategies based on real-time production states, enhancing the algorithm's adaptability to complex production scenarios. The energy-saving scheduling strategy offers a feasible path to reduce machine idle energy consumption and optimize resource utilization, providing practical guidance for companies in achieving green manufacturing and cost control.

Future research could explore the incorporation of additional optimization objectives (such as cost and quality) and improve the algorithms robustness against uncertainties like machine failures and demand fluctuations.

REFERENCES

- [1] Q. Luo, Q. Deng, G. Gong, L. Zhang, W. Han, and K. Li, "An efficient memetic algorithm for distributed flexible job shop scheduling problem with transfers," *Expert Syst. Appl.*, vol. 160, 2020, Art. no. 113721.
- [2] L. Meng, Y. Ren, B. Zhang, J.-Q. Li, H. Sang, and C. Zhang, "MILP modeling and optimization of energy-efficient distributed flexible job shop scheduling problem," *IEEE Access*, vol. 8, pp. 191191–191203, 2020.
- [3] Y. Chen, X. Liao, G. Chen, and Y. Hou, "Dynamic intelligent scheduling in low-carbon heterogeneous distributed flexible job shops with job insertions and transfers," *Sensors*, vol. 24, no. 7, 2024, Art. no. 2251.
- [4] Y. Jia, Y. Zhou, and Y. Fu, "A multi-objective brain storm optimization for integrated distributed flexible job shop and distribution problems," *Heliyon*, vol. 10, no. 16, 2024.
- [5] G. Wang, X. Li, L. Gao, and P. Li, "An effective multi-objective whale swarm algorithm for energy-efficient scheduling of distributed welding flow shop," *Ann. Operations Res.*, vol. 34, pp. 1–33, 2022.
- [6] H. Tang, B. Fang, R. Liu, Y. Li, and S. Guo, "A hybrid teaching and learning-based optimization algorithm for distributed sand casting job-shop scheduling problem," *Appl. Soft Comput.*, vol. 120, 2022, Art. no. 108694.
- [7] K. Zhu et al., "Dynamic distributed flexible job-shop scheduling problem considering operation inspection," *Expert Syst. Appl.*, vol. 224, 2023, Art. no. 119840.
- [8] N. Zhu, G. Gong, D. Lu, D. Huang, N. Peng, and H. Qi, "An effective reformative memetic algorithm for distributed flexible job-shop scheduling problem with order cancellation," *Expert Syst. Appl.*, vol. 237, 2024, Art. no. 121205.
- [9] Z. Pan, D. Lei, and L. Wang, "A knowledge-based two-population optimization algorithm for distributed energy-efficient parallel machines scheduling," *IEEE Trans. Cybern.*, vol. 52, no. 6, pp. 5051–5063, Jun. 2020.
- [10] J. Chang, G. Zhu, H. Zhang, Y. Zhou, X. Luo, and L. Wu, "Reversible data hiding for color images based on adaptive 3D prediction-error expansion and double deep Q-network," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 8, pp. 5055–5067, Aug. 2022.
- [11] A. Kamalova, S. G. Lee, and S. H. Kwon, "Occupancy reward-driven exploration with deep reinforcement learning for mobile robot system," *Appl. Sci.*, vol. 12, no. 18, p. 9249, 2022.
- [12] F. Khoramnejad and M. Erol-Kantarci, "On joint offloading and resource allocation: A double deep q-network approach," *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 4, pp. 1126–1141, Apr. 2021.
- [13] T.-H. S. Li et al., "Fuzzy double deep Q-network-based gait pattern controller for humanoid robots," *IEEE Trans. Fuzzy Syst.*, vol. 30, no. 1, pp. 147–161, Jan. 2020.
- [14] H. Wang, B. R. Sarker, J. Li, and J. Li, "Adaptive scheduling for assembly job shop with uncertain assembly times based on dual Q-learning," *Int. J. Prod. Res.*, vol. 59, no. 19, pp. 5867–5883, 2021.
- [15] F. Zhao, F. Yin, L. Wang, and Y. Yu, "A co-evolution algorithm with dueling reinforcement learning mechanism for the energy-aware distributed heterogeneous flexible flow-shop scheduling problem," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 55, no. 3, pp. 1794–1809, Mar. 2025.
- [16] J.-P. Huang, L. Gao, and X.-Y. Li, "An end-to-end deep reinforcement learning method based on graph neural network for distributed job-shop scheduling problem," *Expert Syst. Appl.*, vol. 238, 2024, Art. no. 121756.
- [17] G. Al Aqel, X. Li, and L. Gao, "A modified iterated greedy algorithm for flexible job shop scheduling problem," *Chin. J. Mech. Eng.*, vol. 32, no. 1, 2019, Art. no. 21.
- [18] M. Raghuvanshi and O. Kakde, "Genetic algorithm with species and sexual selection," in *Proc. IEEE Conf. Cybern. Intell. Syst.* Piscataway, NJ, USA: IEEE Press, 2006, pp. 1–8.
- [19] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [20] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn. PMLR*, 2016, pp. 1995–2003.
- [21] R. Li, "Model-driven learning modeling algorithm for solving the green distributed flexible job shop scheduling problem," M.S. China Univ. Geosciences, 2023. [Online]. Available: <https://link.cnki.net/doi/10.27492/d.cnki.gzdz.2023.000260>
- [22] L. While, P. Hingston, L. Barone, and S. Huband, "A faster algorithm for calculating hypervolume," *IEEE Trans. Evol. Comput.*, vol. 10, no. 1, pp. 29–38, Jan. 2006.
- [23] C. C. Coello and M. S. Lechuga, "MOPSO: A proposal for multiple objective particle swarm optimization," in *Proc. Congr. Evol. Computat. CEC02 (Cat. No. 02TH8600)*, vol. 2. Piscataway, NJ, USA: IEEE Press, 2002, pp. 1051–1056.
- [24] P. Jangir and N. Jangir, "Non-dominated sorting whale optimization algorithm (NSWOA): A multi-objective optimization algorithm for solving engineering design problems," *Glob. J. Res. Eng.*, vol. 17, pp. 15–42, 2017.
- [25] S. Mirjalili, S. Saremi, S. M. Mirjalili, and L. D. S. Coelho, "Multi-objective grey wolf optimizer: A novel algorithm for multi-criterion optimization," *Expert Syst. Appl.*, vol. 47, pp. 106–119, 2016.



Lijun Wang received the B.S. degree in mechatronics engineering from the North University of China, Taiyuan, China, in 1993, the M.S. degree in agricultural mechanization engineering from Henan Agricultural University, Zhengzhou, China, in 2003, and the Ph.D. degree in signal and information processing from PLA Information Engineering University, Zhengzhou, in 2010.

Her research interests include signal processing, image processing, and machine learning, fault diagnosis, and intelligent control.



Chengguang Wang received the B.S. degree in mechanical design, manufacture, and automation in 2021 from the North China University of Water Resources and Electric Power, Zhengzhou, China, where he is currently working toward the Ph.D. degree in management science and engineering with the School of Management and Economics.

His research interests include deep learning, intelligent optimization algorithms, and industrial engineering.



Xiangyang Li received the B.S. degree in automotive engineering from Henan Polytechnic University, Henan, China, in 2019. He is currently working toward the Ph.D. degree management science and engineering with the North China University of Water Resources and Electric Power, Zhengzhou, China.

His research interests include smart manufacturing and system simulation.



Ruixue Cheng received the Ph.D. degree in gas-solids flow measurement from the University of Teesside, Middlesbrough, U.K.

She is a Senior Lecturer and currently with the School of Computing, Engineering and Digital Technologies, Teesside University, Middlesbrough, U.K. Her research interests include sensing techniques and measurement technology development, signal processing, and mathematical modeling.



Xiao Ma received the B.E. and M.E. degrees in mechanical engineering and automation from the North China University of Water Resources and Electric Power, Henan, China, and the Ph.D. degree in mechanical and electrical engineering from Shihezi University, Xinjiang, China.

She is a Lecturer with the School of Mechanical Engineering, North China University of Water Resources and Electric Power. Her research interests include optimization algorithms and smart manufacturing.



Dong Liu received the master's degree in mechanical engineering, North China University of Water Conservancy and Electric Power, Zhengzhou, China, and the Ph.D. degree in power machinery and engineering from Beijing Institute of Technology, Beijing, China.

His research interests include the structural design of diesel combustion chambers, optimization of combustion performance, and reduction of particulate emissions.